

SESSION 8 — String Operations in Pandas

Why String Operations Are Important

In real-world datasets, textual data is very common:

- Product names
- Customer names
- Categories
- Emails

However, text data is often **inconsistent**:

Example:

"Electronics"

"electronics"

"ELECTRONICS"

This creates problems in analysis.

Solution

Pandas provides **vectorized string operations** using:

```
df["column"].str
```

This allows us to apply string functions on entire columns efficiently.

Key Concept

We do NOT use normal Python string methods directly.

Instead we use:

.str accessor

Example:

```
df["column"].str.lower()
```

- ✓ 1. str.lower() and str.upper()

Purpose

Standardize text data.

Syntax

```
df["column"].str.lower()
df["column"].str.upper()
```

Parameters

No required parameters.

Use Cases

- Normalize categories
- Case-insensitive comparison
- Data cleaning

```
import pandas as pd

data = {
    "Category": ["Electronics", "electronics", "ELECTRONICS", "Clothing"]
}

df = pd.DataFrame(data)

# Convert to lowercase
df["Category_lower"] = df["Category"].str.lower()

# Convert to uppercase
df["Category_upper"] = df["Category"].str.upper()

df
```

	Category	Category_lower	Category_upper	
0	Electronics	electronics	ELECTRONICS	
1	electronics	electronics	ELECTRONICS	
2	ELECTRONICS	electronics	ELECTRONICS	
3	Clothing	clothing	CLOTHING	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

∨

2. str.replace()

Purpose

Replace specific text within strings.

Syntax

```
df["column"].str.replace(old, new, regex=True)
```

Parameters

old → text to replace

new → replacement text

regex → whether to treat pattern as regex (default=True)

Use Cases

- Cleaning unwanted characters
- Standardizing names
- Removing symbols

```
df["Category_clean"] = df["Category"].str.replace("Electronics", "Electronic  
df
```

	Category	Category_lower	Category_upper	Category_clean	
0	Electronics	electronics	ELECTRONICS	Electronic Items	
1	electronics	electronics	ELECTRONICS	electronics	
2	ELECTRONICS	electronics	ELECTRONICS	ELECTRONICS	
3	Clothing	clothing	CLOTHING	Clothing	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

✓ 3. str.contains()

Purpose

Check if a substring exists in each element.

Syntax

```
df["column"].str.contains(pattern)
```

Parameters

pattern → substring or regex

case → True/False (case sensitivity)

na → how to handle NaN values

Returns

Boolean Series

Use Case

Filtering rows based on text

```
df[df["Category"].str.contains("Electronics", case=False)]
```

	Category	Category_lower	Category_upper	Category_clean	
0	Electronics	electronics	ELECTRONICS	Electronic Items	
1	electronics	electronics	ELECTRONICS	electronics	
2	ELECTRONICS	electronics	ELECTRONICS	ELECTRONICS	

✓ 4. str.split()

Purpose

Split strings into multiple parts.

Syntax

```
df["column"].str.split(separator)
```

Parameters

separator → character to split on

Use Cases

- Extract first name / last name
- Split categories
- Parse structured text

```
data = {
    "Name": ["Aman Sharma", "Priya Patel", "Rahul Verma"]
}

df = pd.DataFrame(data)

df["Split_Name"] = df["Name"].str.split(" ")

df
```

	Name	Split_Name
0	Aman Sharma	[Aman, Sharma]
1	Priya Patel	[Priya, Patel]
2	Rahul Verma	[Rahul, Verma]

✓ Demo — Normalize Product Categories

Goal:

Convert all category names to lowercase and remove inconsistencies.

```
data = {
    "Category": ["Electronics", "electronics", "ELECTRONICS", "Clothing"]
}

df = pd.DataFrame(data)

df["Category"] = df["Category"].str.lower()

df
```

	Category
0	electronics
1	electronics
2	electronics
3	clothing

SESSION 9 — Date-Time Handling in Pandas

Why Date-Time Handling is Important

Many datasets include date-related information:

- Orders
- Subscriptions
- Transactions

Example:

"2024-01-15"

But these are often stored as **strings**, not actual dates.

Solution

Use:

```
pd.to_datetime()
```

to convert strings into datetime format.

✓ 1. pd.to_datetime()

Purpose

Convert data into datetime format.

Syntax

```
pd.to_datetime(column)
```

Parameters

column → column or list of dates

format → specify date format (optional)

errors → 'raise', 'coerce', 'ignore'

Use Case

- Convert string dates to proper datetime objects

```
data = {  
    "Date": ["2024-01-01", "2024-02-15", "2024-03-10"]  
}  
  
df = pd.DataFrame(data)  
  
df["Date"] = pd.to_datetime(df["Date"])  
  
df
```

	Date	
0	2024-01-01	
1	2024-02-15	
2	2024-03-10	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

2. Extracting Date Components

Once converted, we can extract:

- Year
- Month
- Day

Syntax

```
df["Date"].dt.year  
df["Date"].dt.month  
df["Date"].dt.day
```

```
df["Year"] = df["Date"].dt.year  
df["Month"] = df["Date"].dt.month  
df["Day"] = df["Date"].dt.day  
  
df
```

	Date	Year	Month	Day
0	2024-01-01	2024	1	1
1	2024-02-15	2024	2	15
2	2024-03-10	2024	3	10

3. Working with Date Ranges

Function

`pd.date_range()`

Syntax

`pd.date_range(start, end, freq)`

Parameters

start → start date

end → end date

freq → frequency ('D', 'M', 'Y')

Use Cases

- Generate time series data
- Simulate datasets

```
dates = pd.date_range(start="2024-01-01", end="2024-01-07", freq="D")
```

```
dates
```

```
DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04',  
              '2024-01-05', '2024-01-06', '2024-01-07'],  
              dtype='datetime64[ns]', freq='D')
```

```
dates = pd.date_range(start="2024-01-01", end="2024-12-31", freq="ME")
```

```
dates
```

```
DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-31', '2024-04-30',  
              '2024-05-31', '2024-06-30', '2024-07-31', '2024-08-31',  
              '2024-09-30', '2024-10-31', '2024-11-30', '2024-12-31'],  
              dtype='datetime64[ns]', freq='ME')
```

Demo — Calculate Subscription Tenure

Goal:

Find how long a customer has been subscribed.


```
data = {
    "Start_Date": ["2023-01-01", "2023-06-01"],
    "End_Date": ["2024-01-01", "2024-06-01"]
}

df = pd.DataFrame(data)

df["Start_Date"] = pd.to_datetime(df["Start_Date"])
df["End_Date"] = pd.to_datetime(df["End_Date"])

df["Tenure_Days"] = (df["End_Date"] - df["Start_Date"]).dt.days

df
```

	Start_Date	End_Date	Tenure_Days	
0	2023-01-01	2024-01-01	365	
1	2023-06-01	2024-06-01	366	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

Summary

SESSION 8 — String Operations

- str.lower()
- str.upper()
- str.replace()
- str.contains()
- str.split()

SESSION 9 — Date-Time Handling

- pd.to_datetime()
- Extract year/month/day
- pd.date_range()
- Date difference calculations

Pandas Practice Set — String Operations & Date-Time

This set covers:

- String operations (str.lower, str.upper, replace, contains, split)
- Date-time handling (to_datetime, dt extraction, date_range)

- Real-world data cleaning scenarios

Question 1

Create a DataFrame:

Name = ["Aman Sharma", "Priya Patel", "Rahul Verma"]

Tasks:

- Convert all names to lowercase
 - Convert all names to uppercase
-

Question 2

Using the same dataset:

- Split names into first and last names
 - Store them in two separate columns
-

Question 3

Create a column:

Category = ["Electronics", "electronics", "ELECTRONICS", "Clothing"]

Tasks:

- Normalize all values to lowercase
 - Count how many times "electronics" appears
-

Question 4

Create a column:

Product = ["Phone-123", "Laptop-456", "Tablet-789"]

Tasks:

- Remove numbers using str.replace()
 - Keep only product names
-

Question 5

Create a column:

Email = ["aman@gmail.com", "priya@yahoo.com", "rahul@gmail.com"]

Tasks:

- Filter only Gmail users using str.contains()
-

Question 6

Create a column:

Text = ["apple orange", "banana mango", "grape apple"]

Tasks:

- Check which rows contain the word "apple"
 - Return only those rows
-

Question 7

Create a column:

City = ["Delhi", "Mumbai", None, "Bangalore"]

Tasks:

- Replace missing values with "Unknown"
 - Convert all values to uppercase
-

Question 8

Create a column:

Date = ["2024-01-01", "2024-02-15", "2024-03-10"]

Tasks:

- Convert the column to datetime
 - Extract year and month
-

Question 9

Using the same dataset:

- Extract only the day
 - Create a new column "Day_Name" (optional: weekday)
-

Question 10

Create a date range from:

Start = "2024-01-01"

End = "2024-01-05"

Tasks:

- Generate dates using `pd.date_range()`
- Convert it into a DataFrame

Question 11

Create a dataset:

Start_Date = ["2023-01-01", "2023-06-01"]

End_Date = ["2024-01-01", "2024-06-01"]

Tasks:

- Convert both columns to datetime
 - Calculate tenure in days
-

Question 12

Create a dataset:

Date = ["2024-01-01", "invalid", "2024-03-10"]

Tasks:

- Convert to datetime using errors="coerce"
 - Identify invalid dates
-

Question 13

Create a dataset:

Product = ["Mobile Phone", "Laptop", "Phone Case"]

Tasks:

- Filter rows where the word "Phone" exists
 - Ignore case sensitivity
-

Question 14

Create a dataset:

Full_Name = ["Aman Sharma", "Priya Patel", "Rahul Verma"]

Tasks:

- Extract only last names
 - Create a new column "Last_Name"
-

Question 15 — Data Cleaning + Time Analysis

Create a dataset:

Customer, Category, Start_Date, End_Date

Include:

- Mixed case category values
- Missing values in Category
- Date columns as strings

Tasks:

1 Clean Category column

- Convert to lowercase
- Replace missing values with "unknown"

2 Convert Start_Date and End_Date to datetime

3 Calculate tenure in days

4 Extract Year and Month from Start_Date

5 Filter customers with tenure > 200 days

6 Identify rows where Category contains "electronics"

Start coding or [generate](#) with AI.